

 CODEX · GITHUB · VERCEL · ТЕРМИНАЛ

Словарь выживания в Codex

Не академический справочник, а человеческий перевод технических слов, которые ты будешь слышать в Codex, GitHub, Vercel, терминале и на занятиях.



Что это

Перевод термина на рабочий язык без занудства.



Где увидишь

Codex, терминал, GitHub, Vercel, настройки проекта.



Где риск

Типичная ошибка новичка и зона, где легко сломать проект.

Главная мысль: эти слова не нужно зубрить. Их нужно узнавать в работе, понимать на уровне смысла и использовать, чтобы точнее ставить задачи AI-агенту.

00

Карта документа

Сначала база проекта, потом запуск, Git, публикация, данные и проверка качества. Это не теория ради теории, а навигация по словам, которые всплывают в реальной работе с агентом.

01

Основа проекта

Папки, файлы, frontend, backend, HTML, CSS и JavaScript.

02

Запуск и зависимости

Терминал, команды, localhost, порт, npm install и пакеты.

03

Git и GitHub

История изменений, ветки, commit, push, pull request и конфликты.

04

Публикация

Build, deploy, preview, production и домен.

05

Данные и безопасность

API, запросы, JSON, базы данных, авторизация, ключи и env.

06

Проверка качества

Логи, ошибки, тесты и мобильная версия перед словом «готово».



Как пользоваться

Когда Codex, терминал или Vercel пишет непонятное слово, не гугли хаотично. Найди термин здесь, пойми смысл, потом формулируй агенту задачу точнее.

01

Из чего состоит проект

База, без которой Codex будет «что-то менять», а ты не будешь понимать где и зачем.

Проект

БАЗА

ЧТО ЭТО	Папка со всеми файлами будущего сайта или приложения.
ГДЕ	Codex, файловое дерево, GitHub, Vercel.
ЗАЧЕМ	Чтобы все части продукта лежали в одном месте.
ОШИБКА	Менять случайные файлы вне проекта и потом не понимать, что сломалось.

Файл

БАЗА

ЧТО ЭТО	Один кусок проекта: страница, стиль, логика или настройка.
ГДЕ	<code>index.html</code> , <code>styles.css</code> , <code>script.js</code> , <code>package.json</code> .
ЗАЧЕМ	Чтобы понимать, где текст, где внешний вид, где поведение.
ОШИБКА	Просить Codex «переделай все», хотя нужно изменить один файл.

Frontend

ЭКРАН

ЧТО ЭТО	То, что видит пользователь: экран, кнопки, формы, текст.
ГДЕ	Браузер, страница, компоненты интерфейса.
ЗАЧЕМ	Чтобы управлять пользовательским опытом и внешним видом.
ОШИБКА	Думать, что красивый экран уже значит готовый продукт.

Backend

ЛОГИКА

ЧТО ЭТО	Невидимая часть: данные, доступы, серверная логика, интеграции.
ГДЕ	API routes, server functions, базы данных, настройки сервера.
ЗАЧЕМ	Чтобы проект мог хранить данные и делать действия не только на экране.
ОШИБКА	Добавлять backend, не понимая, какие данные станут публичными.

HTML

СТРУКТУРА

ЧТО ЭТО	Скелет страницы: заголовки, блоки, кнопки, поля.
ГДЕ	Простые сайты и шаблоны страниц.
ЗАЧЕМ	Чтобы страница имела смысловую структуру.
ОШИБКА	Пытаться лечить стиль, когда проблема в структуре страницы.

CSS

ВИД

ЧТО ЭТО	Внешний вид: цвета, отступы, размеры, сетки, адаптивность.
ГДЕ	<code>styles.css</code> , Tailwind-классы, стили компонентов.
ЗАЧЕМ	Чтобы продукт выглядел аккуратно, понятно и не разваливался на телефоне.
ОШИБКА	Добавлять эффекты вместо исправления иерархии и отступов.

JavaScript

ПОВЕДЕНИЕ

ЧТО ЭТО	Логика на странице: клики, расчеты, формы, состояния.
ГДЕ	<code>script.js</code> , React-компоненты, обработчики кнопок.
ЗАЧЕМ	Чтобы интерфейс реагировал на действия пользователя.
ОШИБКА	Вставлять код, не проверив главный сценарий руками.

02

Что ты увидишь в терминале

Терминал не враг. Это табло, где проект честно говорит, жив он или сломан.

Терминал

ПРАВДА

ЧТО ЭТО	Место, где проект запускается и показывает ошибки.
ГДЕ	Codex, VS Code, системное окно команд.
ЗАЧЕМ	Чтобы понимать, жив проект или нет.
ОШИБКА	Закрывать терминал, не прочитав красный текст.

Команда

ДЕЙСТВИЕ

ЧТО ЭТО	Короткая инструкция компьютеру: запусти, установи, проверь.
ГДЕ	<code>npm install</code> , <code>npm run dev</code> , <code>git status</code> .
ЗАЧЕМ	Чтобы выполнять действия точно, а не руками по догадке.
ОШИБКА	Запускать команду не в той папке проекта.

localhost

ЛОКАЛЬНО

ЧТО ЭТО	Адрес проекта, который открыт только на твоём компьютере.
ГДЕ	<code>http://localhost:3000</code> , <code>127.0.0.1:5173</code> .
ЗАЧЕМ	Чтобы проверять проект до публикации.
ОШИБКА	Отправлять localhost клиенту как публичную ссылку.

Порт

ВХОД

ЧТО ЭТО	Номер входа, через который открывается локальный проект.
ГДЕ	<code>3000</code> , <code>4173</code> , <code>5173</code> в адресе локального сайта.
ЗАЧЕМ	Чтобы несколько проектов могли работать одновременно.
ОШИБКА	Думать, что другой порт означает другой интернет-сайт.

Dependency / зависимость

ПАКЕТ

ЧТО ЭТО	Готовый кусок кода, который проект использует.
ГДЕ	<code>package.json</code> , <code>node_modules</code> , сообщения npm.
ЗАЧЕМ	Чтобы не писать все с нуля.
ОШИБКА	Удалять зависимости или переносить проект без установки пакетов.

npm install

УСТАНОВКА

ЧТО ЭТО	Команда «поставь все, что нужно проекту для работы».
ГДЕ	Первый запуск проекта или скачивание с GitHub.
ЗАЧЕМ	Чтобы проект получил нужные зависимости.
ОШИБКА	Запускать проект до установки зависимостей.

03

Как не потерять проект

Git нужен не «для программистов», а чтобы видеть изменения, сохранять рабочие точки и не бояться экспериментов.

Git

ИСТОРИЯ

ЧТО ЭТО	История сохранений проекта.
ГДЕ	Codex, GitHub, команды <code>git status</code> , <code>git commit</code> .
ЗАЧЕМ	Чтобы понимать, что изменилось, и не терять рабочую версию.
ОШИБКА	Не сохранять рабочее состояние перед крупными изменениями.

Repository / репозиторий

ПРОЕКТ

ЧТО ЭТО	Проект вместе с его историей изменений.
ГДЕ	GitHub, Codex, Vercel, ссылка вида <code>github.com/user/project</code> .
ЗАЧЕМ	Чтобы хранить проект, переносить его и подключать к деплою.
ОШИБКА	Думать, что репозиторий это только папка без истории.

Clone / клонировать

СКАЧАТЬ

ЧТО ЭТО	Скачать репозиторий с GitHub на компьютер.
ГДЕ	Кнопка Code на GitHub, команда <code>git clone</code> .
ЗАЧЕМ	Чтобы работать с проектом локально, а не только смотреть сайт.
ОШИБКА	Скачать ZIP и ждать, что Git-история тоже работает как обычно.

git status

СТАТУС

ЧТО ЭТО	Команда «покажи, что изменилось в проекте».
ГДЕ	Терминал, Codex, подсказки Git.
ЗАЧЕМ	Чтобы перед сохранением понимать, какие файлы затронуты.
ОШИБКА	Делать commit вслепую, не посмотрев список изменений.

Staging / git add

ВЫБОР

ЧТО ЭТО	Подготовка выбранных изменений к сохранению в commit.
ГДЕ	<code>git add</code> , Source Control, сообщения staged changes.
ЗАЧЕМ	Чтобы сохранить именно нужные файлы, а не случайный мусор.
ОШИБКА	Добавлять все подряд, включая секреты, временные файлы и черновики.

Commit

СОХРАНЕНИЕ

ЧТО ЭТО	Осмысленная точка сохранения проекта.
ГДЕ	История Git, GitHub, сообщения Codex.
ЗАЧЕМ	Чтобы вернуться к понятному состоянию.
ОШИБКА	Делать commit с названием «fix» и не понимать, что там было.

03

Ветки, push и конфликты

Это слой командной работы: где лежит основная версия, куда отправлять изменения и как не перепутать pull с pull request.

Branch / ветка

ЛИНИЯ

- ЧТО ЭТО** Отдельная линия работы, чтобы не ломать основную версию.
- ГДЕ** GitHub, Codex, Pull Request.
- ЗАЧЕМ** Чтобы пробовать изменения безопаснее.
- ОШИБКА** Путать ветку с копией проекта на компьютере.

Main / основная ветка

ГЛАВНАЯ

- ЧТО ЭТО** Главная линия проекта, где должна лежать рабочая версия.
- ГДЕ** GitHub branch selector, `main`, Vercel production deploy.
- ЗАЧЕМ** Чтобы понимать, какая версия считается основной.
- ОШИБКА** Делать рискованные эксперименты прямо в main без сохранения.

Remote / origin

СВЯЗЬ

- ЧТО ЭТО** Связь локального проекта с репозиторием на GitHub.
- ГДЕ** `origin`, `git remote`, ошибки при push/pull.
- ЗАЧЕМ** Чтобы Git знал, куда отправлять и откуда забирать изменения.
- ОШИБКА** Думать, что GitHub подключен, если remote не настроен.

Pull

ЗАБРАТЬ

- ЧТО ЭТО** Забрать свежие изменения из GitHub к себе.
- ГДЕ** `git pull`, Codex, GitHub workflow.
- ЗАЧЕМ** Чтобы локальная версия не отставала.
- ОШИБКА** Путать Pull с Pull Request.

Push

ОТПРАВИТЬ

- ЧТО ЭТО** Отправить свои изменения на GitHub.
- ГДЕ** `git push`, GitHub, Codex после commit.
- ЗАЧЕМ** Чтобы изменения не остались только на компьютере.
- ОШИБКА** Думать, что commit уже автоматически отправил проект в GitHub.

Pull Request

ПРЕДЛОЖЕНИЕ

- ЧТО ЭТО** Предложение добавить изменения в основную версию проекта.
- ГДЕ** GitHub, командная работа, review.
- ЗАЧЕМ** Чтобы проверить изменения перед объединением.
- ОШИБКА** Думать, что Pull Request забирает изменения к тебе. Это делает Pull.

Merge / объединить

СЛИЯНИЕ

- ЧТО ЭТО** Добавить изменения из одной ветки в другую.
- ГДЕ** GitHub Pull Request, `git merge`, кнопка Merge.
- ЗАЧЕМ** Чтобы безопасно перенести готовую работу в основную версию.
- ОШИБКА** Объединять изменения, не проверив, что проект запускается.

Merge conflict / конфликт

СПОР

- ЧТО ЭТО** Ситуация, когда Git не понимает, какую версию файла оставить.
- ГДЕ** GitHub, Codex, сообщения conflict, маркеры `<<<<<<<`.
- ЗАЧЕМ** Чтобы вручную выбрать правильный вариант изменений.
- ОШИБКА** Удалять конфликтные маркеры наугад и не проверять проект после этого.

04

От README до production

Публикация начинается не с кнопки Deploy. Сначала нужно понять, что проект описан, собран и проверен.

README

ИНСТРУКЦИЯ

ЧТО ЭТО	Главный текстовый файл с описанием проекта.
ГДЕ	<code>README.md</code> на GitHub и в папке проекта.
ЗАЧЕМ	Чтобы быстро понять, что это за проект и как его запустить.
ОШИБКА	Оставлять README пустым, а потом забывать команды запуска.

.gitignore

ФИЛЬТР

ЧТО ЭТО	Файл со списком того, что Git не должен отправлять в репозиторий.
ГДЕ	<code>.gitignore</code> , GitHub, проекты на Node.js, React, Vite.
ЗАЧЕМ	Чтобы не публиковать мусор, сборки, зависимости и секреты.
ОШИБКА	Отправить <code>.env</code> , токены или <code>node_modules</code> в GitHub.

Rollback

НАЗАД

ЧТО ЭТО	Возврат к прошлой рабочей версии.
ГДЕ	Git, Vercel, деплой-платформы.
ЗАЧЕМ	Чтобы быстро откатиться, если новая версия сломалась.
ОШИБКА	Откатывать вслепую и терять чужие изменения.

Build

СБОРКА

ЧТО ЭТО	Сбор проекта в готовую версию для публикации.
ГДЕ	Vercel, Netlify, команды <code>npm run build</code> .
ЗАЧЕМ	Чтобы проверить, что проект готов к деплою.
ОШИБКА	Считать, что локальный запуск заменяет production build.

Deploy / деплой

ПУБЛИКАЦИЯ

ЧТО ЭТО	Публикация проекта в интернет.
ГДЕ	Vercel, Netlify, Render, Railway.
ЗАЧЕМ	Чтобы проект можно было открыть по ссылке.
ОШИБКА	Деплоить проект с секретами, тестовыми платежами и непроверенной формой.

Preview

ТЕСТ

ЧТО ЭТО	Тестовая ссылка для проверки перед настоящей публикацией.
ГДЕ	Vercel preview deployment.
ЗАЧЕМ	Чтобы показать и проверить, не трогая production.
ОШИБКА	Считать preview финальной версией для клиентов.

Production

ОСТОРОЖНО

ЧТО ЭТО	Настоящая публичная версия проекта.
ГДЕ	Vercel production, основной домен.
ЗАЧЕМ	Чтобы проект был доступен реальным пользователям.
ОШИБКА	Выкатывать в production без проверки, аудита секретов и понимания рисков.

Домен

АДРЕС

ЧТО ЭТО	Человеческий адрес сайта, например <code>neovida.ai</code> .
ГДЕ	Настройки Vercel, регистратор домена, DNS.
ЗАЧЕМ	Чтобы не отправлять людям длинную техническую ссылку.
ОШИБКА	Менять DNS, не понимая, куда указывает домен.

05

Где начинается ответственность

Как только проект говорит с сервисами, хранит данные или пускает людей внутрь, это уже не «просто страничка».

API

СВЯЗЬ

ЧТО ЭТО	Способ одному сервису поговорить с другим.
ГДЕ	OpenAI API, Telegram API, CRM, платежные сервисы.
ЗАЧЕМ	Чтобы проект получал или отправлял данные.
ОШИБКА	Подключать внешний сервис, не понимая лимитов, цены и доступа.

Request / запрос

ТУДА

ЧТО ЭТО	«Отправь данные» или «дай мне данные».
ГДЕ	fetch, API calls, network tab.
ЗАЧЕМ	Чтобы приложение могло общаться с сервером.
ОШИБКА	Отправлять лишние личные данные наружу.

Response / ответ

ОБРАТНО

ЧТО ЭТО	То, что сервис вернул на запрос.
ГДЕ	JSON-ответы, статус 200/404/500.
ЗАЧЕМ	Чтобы показать результат пользователю или обработать ошибку.
ОШИБКА	Показывать пользователю сырой технический ответ.

JSON

ФОРМАТ

ЧТО ЭТО	Формат, в котором приложения часто передают данные.
ГДЕ	API-ответы, настройки, тестовые данные.
ЗАЧЕМ	Чтобы данные были понятны коду.
ОШИБКА	Ломать кавычки и скобки, из-за чего проект перестает читать данные.

База данных

ХРАНИЛИЩЕ

ЧТО ЭТО	Место, где хранятся записи: пользователи, заявки, заметки, заказы.
ГДЕ	Supabase, Neon, Postgres, Firebase.
ЗАЧЕМ	Чтобы данные не исчезали после обновления страницы.
ОШИБКА	Хранить чувствительные данные без прав доступа.

Auth / авторизация

ДОСТУП

ЧТО ЭТО	Кто вошел в проект и что ему можно делать.
ГДЕ	Логин, регистрация, роли, личный кабинет.
ЗАЧЕМ	Чтобы защищать данные и разделять права.
ОШИБКА	Добавлять логин «по-быстрому» без проверки доступа к данным.

API-key / ключ

СЕКРЕТ

ЧТО ЭТО	Секретный пропуск к внешнему сервису.
ГДЕ	OpenAI, Telegram, платежи, карты, CRM.
ЗАЧЕМ	Чтобы сервис понимал, кто делает запрос.
ОШИБКА	Вставлять ключ прямо в публичный код или GitHub.

Env / переменные окружения

СЕКРЕТЫ

ЧТО ЭТО	Место для секретов и настроек вне публичного кода.
ГДЕ	<code>.env</code> , <code>.env.local</code> , Vercel Environment Variables.
ЗАЧЕМ	Чтобы не светить ключи и настройки в коде.
ОШИБКА	Публиковать <code>.env</code> в GitHub или отправлять его в чат.

06

Перед «готово» нужна проверка

Красивый экран еще не доказывает, что проект работает. Сначала проверь запуск, главный сценарий, ошибки, секреты и мобильную версию.

Log / лог

ЗАПИСЬ

ЧТО ЭТО	Запись того, что происходило в проекте.
ГДЕ	Терминал, Vercel logs, console.
ЗАЧЕМ	Чтобы искать причину ошибки по фактам.
ОШИБКА	Игнорировать логи и просить AI угадывать.

Error / ошибка

СИГНАЛ

ЧТО ЭТО	Сообщение о том, что именно пошло не так.
ГДЕ	Терминал, браузерная консоль, build logs.
ЗАЧЕМ	Чтобы чинить источник проблемы, а не гадать.
ОШИБКА	Копировать только последнюю строку без полного контекста.

Test / тест

ПРОВЕРКА

ЧТО ЭТО	Проверка, что часть проекта работает как задумано.
ГДЕ	<code>npm test</code> , CI, проверки Codex.
ЗАЧЕМ	Чтобы не ломать старое при добавлении нового.
ОШИБКА	Считать «у меня открылось» полноценной проверкой всего проекта.

Responsive

МОБИЛКА

ЧТО ЭТО	Чтобы сайт нормально выглядел на телефоне, планшете и компьютере.
ГДЕ	CSS, browser preview, mobile screenshot.
ЗАЧЕМ	Большинство людей откроют страницу не на идеальном экране.
ОШИБКА	Проверять дизайн только на большом мониторе.

ПРАВИЛО ГОТОВНОСТИ

Не верь красивому экрану на слово. Открой проект, пройди главный сценарий и только потом пиши Codex: «проверь, что ошибок, утекших секретов и сломанной мобильной версии не осталось».

>_ Фраза для Codex

«Проверь проект как перед публикацией: запуск, ошибки в терминале, основной сценарий, секреты, адаптивность и список измененных файлов».